

LAPORAN PRAKTIKUM STRUKTUR DATA
PEKAN 6



Oleh: Afif Naufal Zahran

NIM: 2511533009

DOSEN PENGAMPU: DR.WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS

Kata Pengantar

Pedoman ini disusun sebagai rujukan resmi bagi mahasiswa Departemen Informatika dalam penyusunan laporan praktikum pada mata kuliah *Struktur Data*. Dokumen ini tidak hanya memberikan gambaran umum mengenai format penulisan, tetapi juga menguraikan secara rinci sistematika laporan, tata cara penyajian isi, serta contoh penulisan kode program yang dilengkapi dengan referensi ilmiah. Melalui panduan ini, mahasiswa diharapkan mampu menyusun laporan yang tidak sekadar memenuhi aspek administratif, tetapi juga mencerminkan ketelitian, keteraturan, dan penerapan kaidah penulisan akademik pada tingkat dasar. Dengan demikian, laporan praktikum yang dihasilkan dapat berfungsi sebagai media pembelajaran, dokumentasi kegiatan, sekaligus sarana untuk melatih keterampilan menulis ilmiah yang akan bermanfaat dalam jenjang studi selanjutnya.

Padang, 2025

Tim Penyusun

DAFTAR ISI

BAB I	
1.1 Latar Belakang.....	
1.2 Tujuan	
1.3 Manfaat	
BAB II	
2,1 Teori.....	
2,2 Program.....	
BAB III.....	
3,1 Kesimpulan.....	
3,2 Saran	

BAB I

PENDAHULUAN

1.1 Latar Belakang

DLL (Doubly Linked List) merupakan jenis struktur data linked list di mana setiap elemen (node) tidak hanya terhubung ke node berikutnya, tetapi juga ke node sebelumnya.

1.2 Tujuan

Tujuan dari laporan praktikum ini agar pembaca dan penulis dapat mendalami dan memahami tentang DLL serta pembuatan Double Linked di java.

1.3 Manfaat

Manfaat dari praktikum tentang operator agar pembaca dan penulis dapat mendalami dan memahami tentang DLL serta pembuatan DLL di java.

BAB II

PEMBAHASAN

2,1 Teori

DLL (Doubly Linked List) merupakan jenis struktur data linked list di mana setiap elemen (node) tidak hanya terhubung ke node berikutnya, tetapi juga ke node sebelumnya

.

2,2 Program

Pada pekan ke-6 ini kita membuat 6 program termasuk 2 program tugas yakni InsertDLL, HapusDLL, NodeDLL, PenelusuranDLL, dan 2 program tugas yakni Lagu_2511533009 dan Musik_2511533009.

a. InsertDLL_2511533009

```
package pekan6_2511533009;

public class InsertDLL_2511533009 {
    //menambahkan node di awal DLL
    static NodeDLL_2511533009 insertBegin_3009(NodeDLL_2511533009 head_3009,
    int data_3009){
        // buat node baru
        NodeDLL_2511533009 new_node_3009 = new NodeDLL_2511533009(data_3009);
        //jadikan pointer nextnya head
        new_node_3009.next_3009 = head_3009;
        //jadikan pointer prev head ke new_node
        if (head_3009 != null){
            head_3009.prev_3009 = new_node_3009;
        }
        return new_node_3009;
    }
    //fungsi menambahkan node di akhir
    public static NodeDLL_2511533009 insertEnd_3009(NodeDLL_2511533009
    head_3009, int newData_3009){
        // buat node baru
        NodeDLL_2511533009 newNode_3009 = new
        NodeDLL_2511533009(newData_3009);
        //jika dll null jadikan head
        if(head_3009 == null) {
            head_3009 = newNode_3009;
        }
        else {
            NodeDLL_2511533009 curr_3009 = head_3009;
            while(curr_3009.next_3009 != null){
                curr_3009 = curr_3009.next_3009;
            }
            curr_3009.next_3009 = newNode_3009;
        }
    }
}
```

```

        newNode_3009.prev_3009 = curr_3009;
    }
    return head_3009;
}

//fungsi menambhakan node di posisi tertentu
public static NodeDLL_2511533009 insertAtPosition_3009(NodeDLL_2511533009
head_3009, int pos_3009, int new_data_3009){
    //buat node baru
    NodeDLL_2511533009 new_node_3009 = new
NodeDLL_2511533009(new_data_3009);
    if(pos_3009 == 1){
        new_node_3009.next_3009 = head_3009;
        if(head_3009 != null){
            head_3009.prev_3009 = new_node_3009;
        }
        head_3009 = new_node_3009;
        return head_3009;
    }
    NodeDLL_2511533009 curr_3009 = head_3009;
    for (int i_3009 = 1; i_3009 < pos_3009 - 1 && curr_3009 != null;
i_3009++){
        curr_3009 = curr_3009.next_3009;
    }
    if (curr_3009 == null){
        System.out.println("Posisi tidak ada");
        return head_3009;
    }
    new_node_3009.prev_3009 = curr_3009;
    new_node_3009.next_3009 = curr_3009.next_3009;
    curr_3009.next_3009 = new_node_3009;
    if (new_node_3009.next_3009 != null){
        new_node_3009.next_3009.prev_3009 = new_node_3009;
    }
    return head_3009;
}

public static void printList_3009(NodeDLL_2511533009 head_3009){
    NodeDLL_2511533009 curr_3009 = head_3009;
    while (curr_3009 != null){
        System.out.print(curr_3009.data_3009 + " <-> ");
        curr_3009 = curr_3009.next_3009;
    }
    System.out.println();
}

public static void main(String[] args){
    // membuat dll 2 <-> 3 <-> 5;
    NodeDLL_2511533009 head_3009 = new NodeDLL_2511533009(2);
    head_3009.next_3009 = new NodeDLL_2511533009(3);
    head_3009.next_3009.prev_3009 = head_3009;
    head_3009.next_3009.next_3009 = new NodeDLL_2511533009(5);
    //cetak DLL awal
    System.out.print("DLL Awal: ");
    printList_3009(head_3009);
    //tambah 1 di awal
    head_3009 = insertBegin_3009(head_3009,1);
    System.out.print(
        "Simpul 1 ditambah di awal: ");
    printList_3009(head_3009);
    //tambah 6 di akhir

```

```

        System.out.print(
            "simpul 6 dtambah di akhir: ");
        int data_3009 = 6;
        head_3009 = insertEnd_3009(head_3009,data_3009);
        printList_3009(head_3009);
        //menambah node 4 di posisi 4
        System.out.print("tambah node 4 di posisi 4: ");
        int data2_3009 = 4;
        int pos_3009 = 4;
        head_3009 = insertAtPosition_3009(head_3009,pos_3009,data2_3009);
        printList_3009(head_3009);
    }
}

```

Pada kode program ini kita akan membuat fitur untuk menambahkan DLL pada posisi awal dan akhir serta menambahkan data pada posisi ke-n pada DLL yang kita buat serta method untuk menampilkan seluruh data dari DLL tersebut.

Output:

```

DLL Awal: 2 <-> 3 <-> 5 <->
Simpul 1 ditambah di awal: 1 <-> 2 <-> 3 <-> 5 <->
simpul 6 dtambah di akhir: 1 <-> 2 <-> 3 <-> 5 <-> 6 <->
tambah node 4 di posisi 4: 1 <-> 2 <-> 3 <-> 4 <-> 5 <-> 6 <->

```

b. HapusDLL_2511533009

```

package pekan6_2511533009;

public class HapusDLL_2511533009 {
    //fungsi menghapus node awal
    public static NodeDLL_2511533009 delHead_3009(NodeDLL_2511533009
head_3009){
        if (head_3009 == null) {
            return null;}
        NodeDLL_2511533009 temp_3009 = head_3009;
        head_3009 = head_3009.next_3009;
        if (head_3009 != null){
            head_3009.prev_3009 = null;}
        return head_3009;
    }
    //fungsi mehapus di akhir
    public static NodeDLL_2511533009 dellLast_3009(NodeDLL_2511533009
head_3009){
        if (head_3009 == null){
            return null;}
        if (head_3009.next_3009 == null){
            return null;}
        NodeDLL_2511533009 curr_3009 = head_3009;
        while (curr_3009.next_3009 != null){
            curr_3009 = curr_3009.next_3009;
        }
    }
}

```

```

    }
    //update pointer previous ndoe
    if (curr_3009.prev_3009 != null){
        curr_3009.prev_3009.next_3009 = null;
    }
    return head_3009;
}
//fungsi menghapus node posisi tertentu
public static NodeDLL_2511533009 delPos_3009(NodeDLL_2511533009 head_3009,
int pos_3009){
    // jika DLL kosong
    if (head_3009 == null){
        return head_3009;
    }
    NodeDLL_2511533009 curr_3009 = head_3009;
    //telusuri sampai ke node yang akan dihaus
    for (int i_3009 = 1; curr_3009 != null && i_3009 < pos_3009; i_3009++){
        curr_3009 = curr_3009.next_3009;
    }
    //jika posisi tidak ditemukan
    if (curr_3009 == null){
        return null;
    }
    //update pointer
    if (curr_3009.prev_3009 != null){
        curr_3009.prev_3009.next_3009 = curr_3009.next_3009;
    }
    if (curr_3009.next_3009 != null){
        curr_3009.next_3009.prev_3009 = curr_3009.prev_3009;
    }
    //jika yang dihapus head
    if (head_3009 == curr_3009){
        head_3009 = curr_3009.next_3009;
    }
    return head_3009;
}

//fungsi mencetak DLL
public static void printList_3009(NodeDLL_2511533009 head_3009){
    NodeDLL_2511533009 curr_3009 = head_3009;
    while (curr_3009 != null){
        System.out.print(curr_3009.data_3009 + " <-> ");
        curr_3009 = curr_3009.next_3009;
    }
    System.out.println();
}

public static void main(String[] args){
    //buat sebuah DLL
    NodeDLL_2511533009 head_3009 = new NodeDLL_2511533009(1);
    head_3009.next_3009 = new NodeDLL_2511533009(2);
    head_3009.next_3009.prev_3009 = head_3009;
    head_3009.next_3009.next_3009 = new NodeDLL_2511533009(3);
    head_3009.next_3009.next_3009.prev_3009 = head_3009.next_3009;
    head_3009.next_3009.next_3009.next_3009 = new NodeDLL_2511533009(4);
    head_3009.next_3009.next_3009.next_3009.prev_3009 =
head_3009.next_3009.next_3009;
    head_3009.next_3009.next_3009.next_3009.next_3009 = new
NodeDLL_2511533009(5);
    head_3009.next_3009.next_3009.next_3009.next_3009.prev_3009 =
head_3009.next_3009.next_3009.next_3009;

```



```

        System.out.println("DLL Awal: ");
        printList_3009(head_3009);

        System.out.println("Setelah Head dihapus: ");
        head_3009 = delHead_3009(head_3009);
        printList_3009(head_3009);

        System.out.println("Setelah node terakhir dihapus: ");
        head_3009 = delLast_3009(head_3009);
        printList_3009(head_3009);

        System.out.println("menghapus node ke 2: ");
        head_3009 = delPos_3009(head_3009,2);

        printList_3009(head_3009);
    }
}

```

Pada program ini terdapat method untuk menghapus data dll pada posisi ke sekian dan program untuk menghapus data kepala (awal) serta data terakhir, dan juga terdapat method untuk menampilkan seluruh data dari DLL.

Output:

```

DLL Awal:
1 <-> 2 <-> 3 <-> 4 <-> 5 <->
Setelah Head dihapus:
2 <-> 3 <-> 4 <-> 5 <->
Setelah node terakhir dihapus:
2 <-> 3 <-> 4 <->
menghapus node ke 2:
2 <-> 4 <->

```

c. NodeDLL_2511533009

```

package pekan6_2511533009;

public class NodeDLL_2511533009 {
    // Mendefinisikan kelas Node
    int data_3009;
    NodeDLL_2511533009 next_3009;
    NodeDLL_2511533009 prev_3009;

    // Konstruktor
    public NodeDLL_2511533009(int data_3009) {
        this.data_3009 = data_3009;
        this.next_3009 = null;
        this.prev_3009 = null;
    }
}

```

Pada program berikut merupakan kelas ADT NodeDLL dimana ADT ini memiliki attribute yakni data (integer) dan next dan prev (NodeDLL).

d. PenelusuranDLL_251153309

```
package pekan6_251153309;

public class PenelusuranDLL_251153309 {
    //fungsi penelusuran maju
    static void forwardTraversal_3009(NodeDLL_2511533009 head_3009){
        // memulai penelusuran dari head_3009
        NodeDLL_2511533009 curr_3009 = head_3009;
        //lanjutkan sampai akhir
        while (curr_3009 != null){
            //print data
            System.out.print(curr_3009.data_3009 + " <-> ");
            //pindah ke node berikutnya
            curr_3009 = curr_3009.next_3009;
        }
        //print spasi
        System.out.println();
    }
    //fungsi penelusuran
    static void backwardTraversal_3009(NodeDLL_2511533009 tail_3009){
        // mulai dari akhir
        NodeDLL_2511533009 curr_3009 = tail_3009;
        //lanjut sampai head
        while (curr_3009 != null) {
            // cetak data
            System.out.print(curr_3009.data_3009 + " <-> ");
            //pindah ke node sebelumnya
            curr_3009 = curr_3009.prev_3009;
        }
        //cetak spasi
        System.out.println();
    }

    public static void main(String[] args) {
        // cetak DLL
        NodeDLL_2511533009 head_3009 = new NodeDLL_2511533009(1);
        NodeDLL_2511533009 second_3009 = new NodeDLL_2511533009(2);
        NodeDLL_2511533009 third_3009 = new NodeDLL_2511533009(3);

        head_3009.next_3009 = second_3009;
        second_3009.prev_3009 = head_3009;
        second_3009.next_3009 = third_3009;
        third_3009.prev_3009= second_3009;

        System.out.println("Penelusuran maju:");
        forwardTraversal_3009(head_3009);

        System.out.println("Penelusuran mundur: ");
        backwardTraversal_3009(third_3009);
    }
}
```

```
}
```

Program ini merupakan kode untuk mencari data dari DLL yang kita punya, kita dapat menari data dari head(data awal) hingga ke akhir (data tail) atau dari tail (data terakhir) hingga ke head(data awal).

e. Lagu_2511533009

```
package pekan6_2511533009;

public class Lagu_2511533009 {
    String judul_3009, penyanyi_3009;
    Lagu_2511533009 next_3009, prev_3009;
    public Lagu_2511533009(String judul_3009, String penyanyi_3009){
        this.judul_3009=judul_3009;
        this.penyanyi_3009=penyanyi_3009;
        this.next_3009 = null;
        this.prev_3009 = null;
    }

    public Lagu_2511533009 getNext_3009(){
        return next_3009;
    }
    public Lagu_2511533009 getPrev_3009(){
        return prev_3009;
    }
    public String getJudul_3009(){
        return judul_3009;
    }
    public String getPenyanyi_3009() {
        return penyanyi_3009;
    }

    public void setPenyanyi_3009(String penyanyi_3009) {
        this.penyanyi_3009 = penyanyi_3009;
    }
    public void setJudul_3009(String judul_3009) {
        this.judul_3009 = judul_3009;
    }
    public void setNext_3009(Lagu_2511533009 next_3009) {
        this.next_3009 = next_3009;
    }
    public void setPrev_3009(Lagu_2511533009 prev_3009) {
        this.prev_3009 = prev_3009;
    }
}
```

Kode ini merupakan kode ADT Lagu yang memiliki attribute Judul dan penyanyi (String) serta next dan prev (Lagu).

f. Musik_2511533009

```
package pekan6_2511533009;
```

```

import java.util.Scanner;

public class Musik_2511533009 {
    public static Lagu_2511533009 tambahLagu_3009(Lagu_2511533009
head_3009,Scanner input_3009){
        System.out.print("Masukkan Judul: ");
        String judul_3009 = input_3009.next();
        System.out.print("Masukkan Penyanyi: ");
        String penyanyi_3009 = input_3009.next();
        if (head_3009 == null) return new
Lagu_2511533009(judul_3009,penyanyi_3009);

        Lagu_2511533009 temp_3009 = head_3009;
        while (temp_3009.getNext_3009() != null){
            temp_3009 = temp_3009.getNext_3009();
        }
        temp_3009.setNext_3009(new Lagu_2511533009(
            judul_3009,
            penyanyi_3009
        ));
        temp_3009.getNext_3009().setPrev_3009(temp_3009);
        return head_3009;
    }

    public static Lagu_2511533009 hapusLaguAwal_3009(Lagu_2511533009
lagu_3009){
        if (lagu_3009 == null){
            System.out.println("Musik Kosong!!");
            return null;
        }
        if (lagu_3009.getNext_3009() != null)
            lagu_3009.getNext_3009().setPrev_3009(null);
        else if(lagu_3009.getNext_3009() == null) return null;
        return lagu_3009.getNext_3009();
    }

    public static void tampilMaju_3009(Lagu_2511533009 head_3009){
        if (head_3009 == null){
            System.out.println("Musik Kosong!!");
            return;
        }
        Lagu_2511533009 temp_3009 = head_3009;
        int count_3009 = 1;
        while (temp_3009.getNext_3009() != null){
            System.out.println("urutan ke " + count_3009);
            System.out.println("Judul: "+temp_3009.judul_3009);
            System.out.println("Penyanyi: " + temp_3009.penyanyi_3009);
            temp_3009 = temp_3009.getNext_3009();
            count_3009++;
        }
        System.out.println("urutan ke " + count_3009);
        System.out.println("Judul: "+temp_3009.judul_3009);
        System.out.println("Penyanyi: " + temp_3009.penyanyi_3009);
    }
}

```

```

public static void tampilMundur_3009(Lagu_2511533009 head_3009){
    if (head_3009 == null){
        System.out.println("Musik Kosong!!");
        return;
    }
    Lagu_2511533009 temp_3009 = head_3009;
    int count_3009 = 1;
    while (temp_3009.getNext_3009() != null){
        count_3009++;
        temp_3009 = temp_3009.getNext_3009();
    }
    while(temp_3009.getPrev_3009() != null){
        System.out.println("No. " + count_3009);
        System.out.println("Judul: " + temp_3009.getJudul_3009());
        System.out.println("Penyanyi: " + temp_3009.getPenyanyi_3009());
        temp_3009 = temp_3009.getPrev_3009();
        count_3009--;
    }
    System.out.println("urutan ke " + count_3009);
    System.out.println("Judul: "+temp_3009.judul_3009);
    System.out.println("Penyanyi: " + temp_3009.penyanyi_3009);
}

public static void cariLagu_3009(Lagu_2511533009 head_3009,Scanner
input_3009){
    System.out.println("Cari lagu berdasarkan judul");
    System.out.print("Judul yang dicari: ");
    String judul_3009 = input_3009.next();
    while (head_3009 != null &&
!head_3009.getJudul_3009().equalsIgnoreCase(judul_3009)){
        head_3009 = head_3009.getNext_3009();
    }
    if (head_3009 == null){
        System.out.println("Lagu dengan judul "+ judul_3009 + " TIDAK
dapat ditemukan!");
        return;
    }
    System.out.println("Lagu dengan judul " + judul_3009 +" ditemukan
dimusik!");
}

public static void main(String[] args){
    Lagu_2511533009 head_3009 = null;
    int pilihan_3009 = 0;
    Scanner input_3009 = new Scanner(System.in);
    while (pilihan_3009 != 6){
        System.out.println("=== Playlist Musik NIM: 2411531005 ===\n" +
            "1. Tambah Lagu\n" +
            "2. Hapus Lagu Pertama\n" +
            "3. Lihat Playlist (Maju)\n" +
            "4. Lihat Playlist (Mundur)\n" +
            "5. Cari Lagu\n" +
            "6. Keluar\n");
        System.out.print("Pilihan: ");
        pilihan_3009 = input_3009.nextInt();
    }
}

```

```

switch (pilihan_3009){
    case 1:
        head_3009 = tambahLagu_3009(head_3009,input_3009);
        break;
    case 2:
        head_3009 = hapusLaguAwal_3009(head_3009);
        break;
    case 3:
        tampilMaju_3009(head_3009);
        break;
    case 4:
        tampilMundur_3009(head_3009);
        break;
    case 5:
        cariLagu_3009(head_3009,input_3009);
        break;
    case 6:
        System.out.println("Keluar Dari Program.\nTerimakasih
telah menggunakan!");
        break;
}
}
}
}

```

Pada program ini terdapat method untuk menambahkan, menghapus, menampilkan lagi dari head ke tail dan dari tail ke head, serta mencari lagu dengan struktur data DDL (Double Linked List) dengan memanfaatkan next dan prev dari ADT Lagu.

BAB III

PENUTUPAN

3,1 Kesimpulan

DDL pada bahasa pemrograman baik itu *Java* ataupun bahasa pemograman yang lain karena hal yang sangat fundamental pada bahasa pemrograman dan struktur data.

3,2 Saran

Laporan praktikum ini masih memiliki kekurangan. Karena itu, penulis sangat terbuka terhadap saran dan kritikan agar dapat meningkatkan kualits laporan ini dan laporan-laporan selanjutnya.